PASSWORD MANAGER WITH ENCRYPTION REPORT

INTRODUCTION:

In the modern digital era, managing multiple online accounts securely has become a major challenge. Users often reuse passwords or store them insecurely, leading to potential security risks. A Password Manager is a tool designed to store and manage user credentials securely.

This project focuses on developing a Password Manager with Encryption that ensures all stored passwords are protected using strong encryption techniques. The system uses Fernet encryption from the Python cryptography library to securely store passwords in an encrypted format. Only authorized users can retrieve and decrypt the stored data. This enhances confidentiality and prevents unauthorized access to sensitive information.

OBJECTIVE:

• To develop a secure password manager using Python
• To store passwords in encrypted format
• To implement encryption and decryption using Fernet
• To allow users to add and retrieve passwords securely
• To prevent unauthorized access to stored credentials
• To gain practical knowledge in cybersecurity and encryption

TOOLS USED:

• Kali Linux – used as the development and execution environment
• Python 3 – used to implement the password manager
• Cryptography Library – used for encryption and decryption (Fernet)
• JSON – used to store password data
• GitHub – used to manage and store the project repository

SYSTEM ARCHITECTURE:

• The user runs the password manager tool through the terminal in Kali Linux

• The user enters details such as website, username, and password

• The tool encrypts the password using Fernet encryption and stores it in a file

• When requested, the tool decrypts and displays the password securely

• All operations are handled through a simple menu-based interface

IMPLEMENTATION:

- Initialize Python script and required libraries
- Generate and load encryption key
- Accept user input (site, username, password)
- Encrypt password using Fernet algorithm
- Store encrypted data in a JSON file
- Retrieve stored data when required
- Decrypt password and display output
- Handle errors such as missing data or invalid input

PROCEDURE:

Step 1: Creating the Project Folder and navigating into it

Command: cd SKCT_727823tucy039_PasswordManager

```
┌──(ruban㉿Ruban)-[~]
└─$ cd SKCT_727823tucy039_PasswordManager
```

Step 2: Generating the encryption key (key.key file)

Command: ls

```
┌──(ruban㉿Ruban)-[~/SKCT_727823tucy039_PasswordManager]
└─$ ls
analyze_result.py  key.key  key_gen.py  passwords.json  pipeline_12345.yml  run_tool.py  setup_lab.py  tool_main.py
```

Step 3: Creating and executing the password manager tool

Command: python run_tool.py

```
┌──(ruban㉿Ruban)-[~/SKCT_727823tucy039_PasswordManager]
└─$ python run_tool.py
ROLL: 12345
TIME: 2026-03-29 20:23:10.724149
Running password manager tool ...
```

Step 4: Running test cases (add, view, invalid input)

Command: python tool_main.py

```
┌──(ruban㉿Ruban)-[~/SKCT_727823tucy039_PasswordManager]
└─$ python tool_main.py

1. Save Password
2. View Password
3. Exit
Enter choice:
```

Step 5: Running Pipeline – Setup Stage

Command: python setup_lab.py

```
┌──(ruban㉿Ruban)-[~/SKCT_727823tucy039_PasswordManager]
└─$ python setup_lab.py
ROLL: 12345
TIME: 2026-03-29 21:57:58.053160
Setting up lab environment ...
```

Step 6: Running Pipeline – Tool Execution

Command: python run_tool.py

```
┌──(ruban㉿Ruban)-[~/SKCT_727823tucy039_PasswordManager]
└─$ python run_tool.py
ROLL: 12345
TIME: 2026-03-29 21:58:05.659274
Running password manager tool ...
```

Step 7: Running Pipeline – Result Analysis

Command: python analyze_result.py

```
┌──(ruban㉿Ruban)-[~/SKCT_727823tucy039_PasswordManager]
└─$ python analyze_result.py
ROLL: 12345
TIME: 2026-03-29 21:57:39.752265
Analyzing results ...
```

Step 8: Verifying Output

Command: cat password.json

```
┌──(ruban㉿Ruban)-[~/SKCT_727823tucy039_PasswordManager]
└─$ cat passwords.json
{"2": {"username": "ruban", "password": "gAAAAABpyTqjRkXAqzzpUkuJjX2Kb-uHYM5q-4zLFfwondTLOLYzRY65GGR0mBAIQQSZlHNHxm
5C7hagCWxibG6JE9zufpRrbg="}}
```

TEST CASES AND RESULTS:

Test Case	Description	Result
1	Add Password	Stored Successfully
2	View Password	Decrypted Correctly
3	Invalid Input	Error Handled

ANALYSIS OF RESULTS:

The results show that the password manager successfully stores and retrieves user credentials securely. The encryption mechanism ensures that passwords are not stored in plain text, making it difficult for unauthorized users to access sensitive data.

The tool effectively handles different scenarios such as adding new passwords, retrieving stored passwords, and managing invalid inputs. This demonstrates the importance of encryption in protecting user data and improving system security.

PIPELINE IMPLEMENTATION:

• setup_lab.py – Initializes the environment and prints roll number and timestamp

• run_tool.py – Executes the password manager and performs operations

• analyze_results.py – Displays final output and verifies results

ERROR ENCOUNTERED:

• Error Message: FileNotFoundError: passwords.json not found

• Explanation: This error occurred when the program tried to read the file before it was created

• Solution: The issue was fixed by using a try-except block to handle missing files

ADVANTAGES:

• Provides secure storage of passwords
• Uses strong encryption for data protection
• Easy to use and simple interface
• Prevents unauthorized access
• Helps users manage multiple credentials safely

LIMITATIONS:

• Requires key file for decryption
• If key is lost, passwords cannot be recovered
• Basic implementation without advanced features
• No graphical user interface

CONCLUSION:

The Password Manager with Encryption developed in this project provides a secure way to store and manage user credentials. By using Fernet encryption, the system ensures that passwords are protected from unauthorized access.

The tool successfully demonstrates how encryption can be applied in real-world applications to enhance security. Through different test cases, the system proved its ability to securely store, retrieve, and manage passwords. Overall, this project helps in understanding the importance of data protection and secure programming practices.